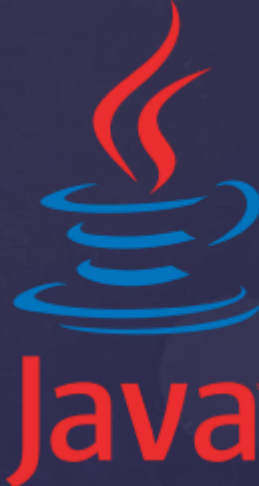




Programmation Orientée Objet



Katy SAINTIN
Octobre 2016



Pourquoi autant de succès ?

Les objets et les classes

Les pointeurs et les références

Les classes

- ✓ *Introduction*
- ✓ *Les champs ou données de la classe*
- ✓ *Les méthodes de classe*
- ✓ ***L'encapsulation***
- ✓ *Le constructeur*
- ✓ *Mot clé « this »*
- ✓ *Les accesseurs*
- ✓ *La méthode toString()*

Pourquoi autant de succès ?

Parce qu'elle nous aide à mieux organiser la complexité inhérente aux logiciels et celle caractéristique du monde réel à modéliser.

Grâce à la POO les logiciels sont :

- Plus maintenables
- Modulaires
- Souples et évolutifs
- Optimisés



procédural



objet

Avant, les développeurs concentraient leurs efforts sur les algorithmes. Avec les objets, le travail sur l'algorithme est minimisé. Leur souplesse permet aux développeurs d'aller beaucoup plus loin dans la réalisation et la conception de programmes "intelligents" et modulaires.

Un objet, c'est une entité réutilisable, qui se caractérise par :

- Un **état** : dans lequel l'objet se trouve.
- Un **comportement** : ce que l'objet est capable de faire.
- Une **identité unique** : ce qui distingue l'objet d'un autre.

Une classe est un **modèle** ou une **maquette** à partir de laquelle les objets vont être construits.

Ne jamais confondre une classe et un objet. L'objet est une **instance** de la classe qui représente son modèle.

Classe

Recette Pizza



Instance 1



Etat : Bien cuite

Identité : Pizza n°1

Comportement : Cuire

Objet

Instance 2



Etat : Crue

Identité : Pizza n°2

Comportement : Cuire

Instance 3



Etat : Cuite

Identité : Pizza n°3

Comportement : Cuire

Rappel : Objet, classe, pointeur et référence

Objet & classe

- ✓ *Objet* : Ma pizza préférée « Bella Italia »
- ✓ *Classe* : Recette de ma pizza

L'objet pizza contient beaucoup d'ingrédient et est donc volumineuse et difficile à recopier.



Pointeur & référence

- ✓ *Pointeur* : Adresse de la pizzeria qui fait ma pizza préférée, facile à recopier.(C++)
- ✓ *Référence* : Nom de la pizzeria « Chez Tony » facile à utiliser.(Java)



Les champs ou données d'une classe

Une classe peut contenir des données qui peuvent être :
des objets ou des types primitifs. Les valeurs affectées à ces champs représente l'état d'un objet.

Exercice écrit à rendre sur feuille :

Prenons, l'exemple d'un objet Pokémon,
lister les champs possibles et leur type
en s'aidant de l'image ci-contre :

- 8 champs sont explicites
- 1 champ est implicite
- 1 champ est utile pour l'IHM*

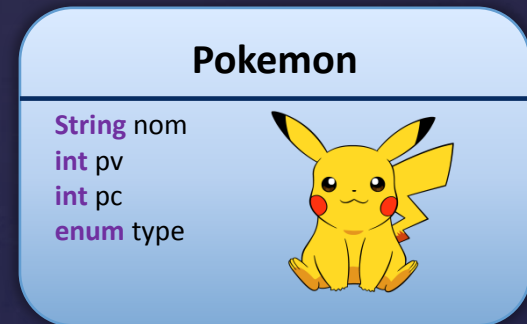


*IHM = Interface Homme Machine

Les champs ou données d'une classe solution

Prenons, l'exemple d'un objet Pokémon, ses attributs sont :

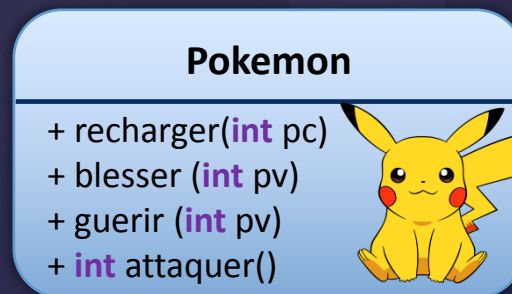
- ✓ **String** nom : un nom
- ✓ **int** pc : ses points de combat
- ✓ **int** pv : ses points de vie
- ✓ **enum** type : un type parmi Normal, Electrik, Eau, Feu ...
- ✓ **int** pvMax : son point de vie maximum
- ✓ **float** poids : son poids en kilos
- ✓ **float** taille : sa taille en mètres
- ✓ **boolean** favori : si le pokemon est un favori
- ✓ **int** numero : son numéro (identifiant)
- ✓ **String** avatar : chemin du fichier image



Ces méthodes sont des fonctions appelables par un client et elles définissent le comportement ou une action de l'objet.

Toujours dans l'exemple de l'objet Pokémon, ce dernier peut :

- ✓ Se recharger : gagner des points de combat,
- ✓ Se blesser : perdre des points de vie
- ✓ Se guérir : gagner des points de vie
- ✓ Attaquer : Infliger des blessures à un autre Pokémon



Les méthodes peuvent accepter des données en paramètres mais peuvent également renvoyer des données. (signature)

L'encapsulation permet de définir des niveaux de visibilité des éléments de la classe. Cet accès aux données se définit par le biais de « modifieurs » parmi les choix suivants :

public : donnée accessible à toutes les classes.

protected : donnée réservée aux classes héritières du même package

private : donnée limitée à la classe elle-même
sans spécification du « modifier » la classe est package **friendly**

Toujours dans l'exemple de l'objet Pokémon, on peut imaginer que :

- ✓ Les champs seront privées et accessibles via des accesseurs
- ✓ Les méthodes seront publiques car accessibles à l'utilisateur

1. Créer une classe Pokemon
2. Créer l'énumération Type {Normal, Electrik, Eau, Feu}
3. Créer les champs :
 - ✓ **int** pv (qui ne peut jamais être négatif)
 - ✓ **int** pc (qui ne peut jamais être négatif)
 - ✓ **String** nom
 - ✓ **Type** type (égale à Normal par défaut)
4. Créer les méthodes :
 - ✓ renommer (String nom) : qui change le nom
 - ✓ recharger(int pcPlus) : qui ajoute pcPlus à pc
 - ✓ guerir(int pvPlus) : qui ajoute pvPlus à pv
 - ✓ blesser(int pvMoins) : qui enlève pvMoins à pv
 - ✓ int Attaquer() : qui retourne 0 par défaut

TP n°1 : Classe Pokemon solution



The value of the field is not used

The assignment has no effect

```
1 package fr.ifa.pokemons;
2
3 public class Pokemon {
4
5     //Enumeration type de pokemon
6     public static enum Type {
7         Normal, Electrik, Eau, Feu
8     }
9
10    private String nom = "";
11    // Point de vie
12    private int pv = 0;
13    // Point de combat
14    private int pc = 0;
15    // Type de pokemon
16    private Type type = Type.Normal;
17
18    public void renommer(String nom){
19        nom = nom;
20    }
21
22    public void recharger(int pcPlus) {
23        pc = pc + pcPlus;
24    }
25
26    public void guerir(int pvPlus) {
27        pv = pv + pvPlus;
28    }
29
30    public void blesser(int pvMoins) {
31        pv = pv + pvMoins;
32    }
33
34    public int Attaquer() {
35        return 0;
36    }
37
38 }
```

Variables de classe

Variable d'instance

Variable locale

Le mot clé "this" sert à désigner l'objet courant de la classe dans laquelle elle est utilisée. On peut distinguer 3 cas d'utilisation :

1. Pour éviter la répétition des variables pour plus de lisibilité (cf TP précédent pour la méthode renommer. On pourrait nommer le paramètre autrement, mais à ce qu'il paraît, les informaticiens sont de gros flemmards. 🤪

```
public void renommer(String nom){  
    this.nom = nom;  
}
```

2. Pour envoyer la référence d'un objet en paramètre.

```
public void renommer(String nom){  
    this.nom = nom;  
    maMethode(this, nom);  
}
```

3. Pour appeler un constructeur de la classe.

```
public Pokemon() {  
    this("Pikachu");  
}  
  
public Pokemon(String nom) {  
    renommer(nom);  
}
```

Un constructeur est une méthode un peu particulière qui :

- ✓ porte le **même nom** que la classe
- ✓ **ne renvoie rien** (pas même void)
- ✓ ne pourra être exécuté **qu'une seule fois**.
- ✓ peut prendre des paramètres
- ✓ peut être surchargé
- ✓ qui se charge d'initialiser les valeurs des champs.

L'appel au constructeur se fait avec l'opérateur **new**, on appelle cela **l'instanciation**.

```
//Constructeur vide
public Pokemon(){
    this("", 0, 0, Type.Normal);
}

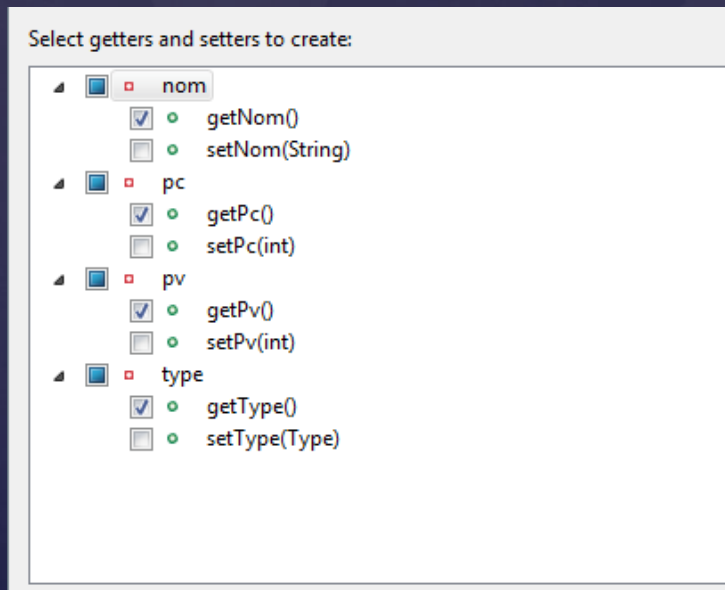
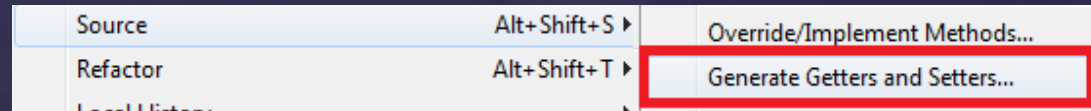
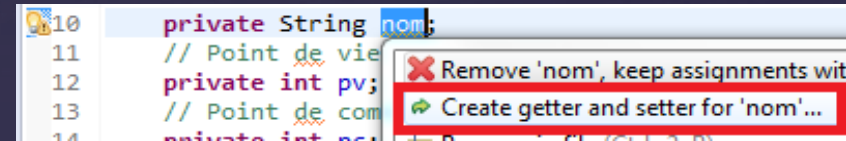
//Constructeur avec paramètre
public Pokemon(String nom, int pv, int pc, Type type){
    this.pv = 0;
    this.pc = 0;
    this.type = type;
    renommer(nom);
}

public static void main(String[] args) {
    //Creation d'un pokemon de base
    Pokemon pokemon = new Pokemon();
    //Creation d'un pokemon Pikachu
    Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
}
```



Les accesseurs sont des méthodes permettant l'accès et la modification des champs de classe (souvent privés), de manière **sécurisée**. Le standard java pose les règles de nommage suivant :

- Pour un champs non booléen maVariable
- La méthode d'accès (getter) sera **get**MaVariable()
- La méthode de modification (setter) sera **set**MaVariable()
- Pour un champs booléen monBooleen
- Le getter sera **is**MonBooleen()



```
public String getNom() {  
    return nom;  
}  
  
public int getPv() {  
    return pv;  
}  
  
public int getPc() {  
    return pc;  
}  
  
public Type getType() {  
    return type;  
}
```

La méthode toString()

La méthode `toString()` est une méthode de la classe `Objet` dont toutes classes *dérivent*. Elle est utilisée dans de nombreuses API java, en particulier la plus utile pour l'affichage d'un objet à savoir `System.out.println`.

- ✓ Sans *surcharge* la classe renvoie l'adresse de l'objet.
- ✓ Sinon le développeur peut spécifier le comportement souhaité (*polymorphisme*).

```
67 public static void main(String[] args) {
68     //Creation d'un pokemon de base
69     Pokemon pokemon = new Pokemon();
70     //Creation d'un pokemon Pikachu
71     Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
72
73     System.out.println("pokemon=" + pokemon);
74     System.out.println("pikachu=" + pikachu);
75 }
```

Console Problems

<terminated> Pokemon [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (9 oct. 2016)

pokemon=fr.ifa.pokemons.Pokemon@2a139a55
pikachu=fr.ifa.pokemons.Pokemon@15db9742

```
67 @Override
68 public String toString() {
69     String toString = "Nom="+getNom() + "\n";
70     toString = toString + "Type="+getType() + "\n";
71     toString = toString + "Pc="+getPc() + "\n";
72     toString = toString + "Pv="+getPv() + "\n";
73     return toString;
74 }
75
76 public static void main(String[] args) {
77     //Creation d'un pokemon de base
78     Pokemon pokemon = new Pokemon();
79     //Creation d'un pokemon Pikachu
80     Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
81
82     System.out.println(pokemon);
83     System.out.println(pikachu);
84 }
```

Console Problems

<terminated> Pokemon [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (9 oct. 2016)

Nom=
Type=Normal
Pc=0
Pv=0

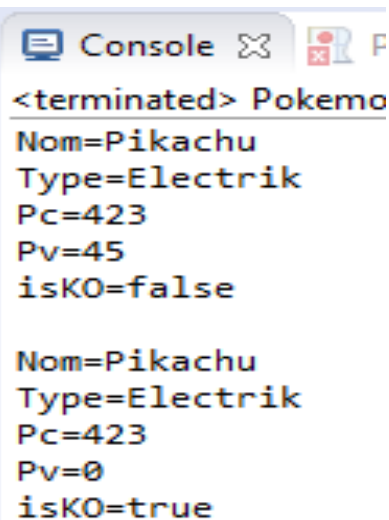
Nom=Pikachu
Type=Electrik
Pc=0
Pv=0

TP n°2 : Classe Pokemon avancée

1. Utiliser le mot clé **this** dans la méthode renommer
2. Rajouter un champs pvMax de type int
3. Créer des getter pour nom, pc, pv, type
4. Créer un constructeur avec nom, pc, pv et type en paramètre (utiliser le mot clé **this** pour affecter les champs)
5. Affecter pvMax = pv dans le constructeur
6. Faire en sorte que pv ne soit jamais négatif dans la méthode blesser.
7. Faire en sorte que pv ne soit jamais supérieur à pvMax dans la méthode guérir.
8. Rajouter une méthode isKO() qui retourne true si pv = 0
9. Surcharger la méthode toString afin d'afficher les valeurs de champs suivants : nom, pv, pc, type et isKO
10. Créer un main et instancier la classe Pokemon au choix si vous avez POKEMON GO, sinon reprendre l'exemple de Pikachu.

TP n°2 : Classe Pokemon avancée solution

```
1 package fr.ifa.pokemons;
2
3 public class Pokemon {
4
5     //Enumeration type de pokemon
6     public static enum Type {
7         Normal, Electrik, Eau, Feu
8     }
9
10    private String nom;
11    // Point de vie
12    private int pv;
13    // Point de combat
14    private int pc;
15    // Type de pokemon
16    private Type type;
17    // pvMax
18    private int pvMax;
19
20    //Constructeur avec paramètre
21    public Pokemon(final String nom, final int pv, final int pc, final Type type){
22        this.pv = pv;
23        this.pc = pc;
24        this.type = type;
25        this.pvMax = pv;
26        renommer(nom);
27    }
28
29    public String getNom() {
30        return nom;
31    }
32
33    public int getPv() {
34        return pv;
35    }
36
37    public int getPc() {
38        return pc;
39    }
40
41    public Type getType() {
42        return type;
43    }
44
```



```
<terminated> Pokemo
Nom=Pikachu
Type=Electrik
Pc=423
Pv=45
isKO=false

Nom=Pikachu
Type=Electrik
Pc=423
Pv=0
isKO=true
```

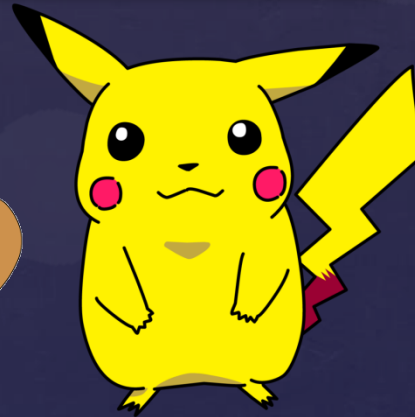
```
45    public void renommer(String nom){
46        if(nom != null && !nom.isEmpty()){
47            this.nom = nom;
48        }
49    }
50
51    public void guerir(int pvPlus) {
52        pv = pv + pvPlus > pvMax ? pvMax : pv;
53    }
54
55    public void blesser(int pvMoins) {
56        pv = pv - pvMoins < 0 ? 0 : pv;
57    }
58
59    public boolean isKO(){
60        return (getPv() == 0);
61    }
62
63    @Override
64    public String toString() {
65        String toString = "Nom="+getNom() + "\n";
66        toString = toString + "Type="+getType() + "\n";
67        toString = toString + "Pc="+getPc() + "\n";
68        toString = toString + "Pv="+getPv() + "\n";
69        toString = toString + "isKO="+isKO() + "\n";
70        return toString;
71    }
72
73
74    public static void main(String[] args) {
75        //Creation d'un pokemon Pikachu
76        Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
77        System.out.println(pikachu);
78        //Blesser pikachu
79        pikachu.blesser(100);
80        System.out.println(pikachu);
81    }

```

- ✓ *Classes enfants*
- ✓ *Classe parent*
- ✓ *Transtypage (Casting)*
- ✓ *Empêcher l'héritage*
 - *Modificateur final pour une classe*
 - *Modificateur final pour une méthode*
 - *Modificateur final pour un champ*
- ✓ *Mécanisme de l'héritage*
 - *La classe Objet et chaîne d'héritage*
 - *Masquage ou redéfinition de méthodes*
 - *Surcharge de méthodes*
 - *Liaisons statiques ou dynamiques*
 - *Polymorphisme*

Classes enfants et classe parent

L'héritage vous permettra d'enrichir des classes existantes qui hériteront du savoir-faire de leur parent. Cette technique permet d'aller beaucoup plus loin dans la résolution de problèmes complexes.

The Pokémon logo is displayed in its characteristic stylized, bubbly font. The letters are yellow with a thick blue outline, set against a light blue rounded rectangular background.

Dans cet exemple les classes **filles** ou **enfants** Têtarte, Miaouss, Pikachu, Salamèche héritent de la classe **mère** ou **parent** Pokémon. La classe parent est aussi appelée la **superclasse**.

Toutes les classes en java dérivent de la classe Object.

```
public class Object {  
    public Object() {...} // constructeur  
  
    public String toString() {...}  
  
    protected native Object clone() throws CloneNotSupportedException {...}  
  
    public equals(java.lang.Object) {...}  
    public native int hashCode() {...}  
  
    protected void finalize() throws Throwable {...}  
  
    public final native Class getClass() {...}  
  
    // méthodes utilisées dans la gestion des threads  
    public final native void notify() {...}  
    public final native void notifyAll() {...}  
  
    public final void wait(long) throws InterruptedException {...}  
    public final void wait(long, int) throws InterruptedException {...}  
}
```

toString() utilisée pour System.out et autre composant graphique

clone() pour la duplication de l'objet

equals() pour la comparaison avec un autre objet

hashCode() retourne un numérique utilisé pour equals

Click droit > New Class

Source folder: Pokemons/src Browse...

Package: fr.ifa.pokemons Browse...

☐ Enclosing type: Browse...

Name: Pikachu

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: fr.ifa.pokemons.Pokemon Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☐ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☒ Generate comments

Mot clé **super** fait référence à la superclasse.

Le C++ supporte l'héritage multiple qui permet à une classe d'hériter de plusieurs classes parents. En JAVA, il n'y a pas d'héritage multiple néanmoins, il y a les **interfaces** que nous verrons plus tard.

```
1 package fr.ifa.pokemons;
2
3 public class Pikachu extends Pokemon {
4
5     public Pikachu(int pv, int pc) {
6         super("Pikachu", pv, pc, Type.Electrik);
7     }
8
9     public static void main(String[] args) {
10         //Creation d'un pokemon Pikachu
11         Pokemon pikachu = new Pikachu(45,423);
12         System.out.println(pikachu);
13     }
14 }
```

Console

```
<terminated> Pikachu [Java Application] C:\Program Files\Java\jre1.8.0_60\b
Nom=Pikachu
Type=Electrik
Pc=423
Pv=45
isKO=false
```

Transtypage (casting)

Le transtypage est l'action qui consiste à convertir les objets en objets plus ou moins spécifiques. Le transtypage implique une relation d'héritage entre les classes impliquées.

Il est possible, sous certaines conditions de convertir un Pokemon en Pikachu. Par contre, il est impossible de convertir un Pikachu en Miaouss puisque aucune relation d'héritage existe entre les deux classes.



*transtypage ascendant
ou généralisation
ou surtypage
(upcasting)
implicite*

*transtypage descendant
ou spécialisation
(downcasting)*



```
1 package fr.ifa.pokemons;
2
3 public class Pikachu extends Pokemon {
4
5     public Pikachu(int pv, int pc) {
6         super("Pikachu", pv, pc, Type.Electrik);
7     }
8
9     public void attaquer(){
10         System.out.println("Attaque électrique");
11     }
12
13     public static void main(String[] args) {
14         //Creation d'un pokemon Pikachu
15         Pokemon pokemon = new Pikachu(45,423);
16
17         //Transtypage descendant pour faire appel à la méthode attaquer
18         Pikachu pikachu = (Pikachu)pokemon;
19         pikachu.attaquer();
20
21         //Transtypage ascendant
22         System.out.println(((Pokemon)pikachu).toString());
23     }
24 }
```

Afin de protéger certaines classes et d'empêcher qu'elles puissent avoir des enfants, on utilise le mot clé ***final***. (ex String)

```
public final class MaClassePrivee {
```

Ce principe permet de :

- de protéger ses propres classes
- d'améliorer les performances d'un programme (moins de travail au microprocesseur et contribuera à améliorer les performances)

Lorsque vous appliquez ce modificateur à une méthode mais pas forcément à sa classe, les classes enfant ne pourront en aucun cas surcharger cette méthode.

```
public class MaClassePrivee {  
  
    public final void maMethodePrivee(){  
        System.out.println("Méthode impossible à surcharger !!");  
    }  
}
```


Empêcher la modification d'un champs et le mot clé static

Pour un champ, la signification n'est pas la même. Il désigne un champ qui ne peut être modifié. Vous appliquerez le modificateur *final* aux constantes. Vous veillerez également à mettre en majuscule le nom de ces constantes afin de respecter la convention :

```
public class MaClassePrivee {  
  
    public final String MA_CONSTANTE = "Constante";  
}
```

En général, ces champs constants sont également dits "statiques" grâce à l'attribut static. Lorsque le champs est statique sa valeur et sa référence est partagée par toutes les instances de la classe. L'allocation mémoire ne se fait qu'une seule fois.

```
public class Pokemon {  
  
    //Enumeration type de pokemon  
    public static enum Type {  
        Normal, Electrik, Eau, Feu  
    }  
  
    private static final Type TYPE_PAR_DEFAULT = Type.Normal;  
    // Type de pokemon  
    private Type type = TYPE_PAR_DEFAULT;  
}
```

Pour bien comprendre ce qui se passe lorsqu'un appel de méthode est appliqué à des objets de types différents dans la hiérarchie des classes :

- La sous classe vérifie qu'elle possède bien une méthode de ce nom avec la même signature. Si c'est le cas, elle l'utilise
- Sinon, la classe parent est interrogée et détermine si elle possède une telle méthode. Si c'est le cas elle l'utilise.

Ce processus peut remonter ainsi toute la hiérarchie des classes jusqu'à ce qu'une méthode correspondante soit retrouvée. Si aucune ne correspond, le compilateur génère une erreur.

Exemple d'introspection avec **l'API de réflexion**.

Chaîne d'héritage illustration avec API de réflexion.

```
package fr.ifa.pokemons;

import java.lang.reflect.Field;
import java.lang.reflect.Method;

public class InstrospectionClass {

    public static void main(String[] args) {
        //Recuperation des champs
        Field[] declaredFields = Pikachu.class.getDeclaredFields();
        System.out.println("Nombre de champs pour Pikachu=" + declaredFields.length);
        for (Field field : declaredFields){
            System.out.println(field.getName());
        }

        //Recuperation des champs
        Field[] superDeclaredFields = Pikachu.class.getSuperclass().getDeclaredFields();
        System.out.println("Nombre de champs pour la classe parente de Pikachu=" + superDeclaredFields.length);
        for (Field field : superDeclaredFields){
            System.out.println(field.getName());
        }

        //Recuperation des méthodes
        Method[] declaredMethods = Pikachu.class.getDeclaredMethods();
        System.out.println("Nombre de méthodes pour la classe Pikachu=" + declaredMethods.length);
        for (Method method : declaredMethods) {
            System.out.println(method.getName());
        }

        Method[] superDeclaredMethods = Pikachu.class.getSuperclass().getDeclaredMethods();
        System.out.println("Nombre de méthodes pour la super classe Pikachu=" + superDeclaredMethods.length);
        for (Method method : superDeclaredMethods) {
            System.out.println(method.getName());
        }
    }
}
```

Chaîne d'héritage illustration avec API de réflexion.

Console

<terminated> IntrospectionClass [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (4 nov. 2016 22:57:04)

Nombre de champs pour Pikachu=0

Nombre de champs pour la classe parente de Pikachu=6

TYPE_PAR_DEFAULT

nom

pv

pc

type

pvMax

Nombre de méthodes pour la classe Pikachu=2

main

attaquer

Nombre de méthodes pour la super classe Pikachu=11

main

toString

getType

renommer

getNom

getPv

getPc

guérir

blessier

isKO

recharger

Redéfinition (masquage) des méthodes

Afin de modifier le comportement d'une méthode. On peut redéfinir (Override, masquage) des méthodes dans les classes filles en redéclarant une méthode qui porte **le même nom et la même signature**.

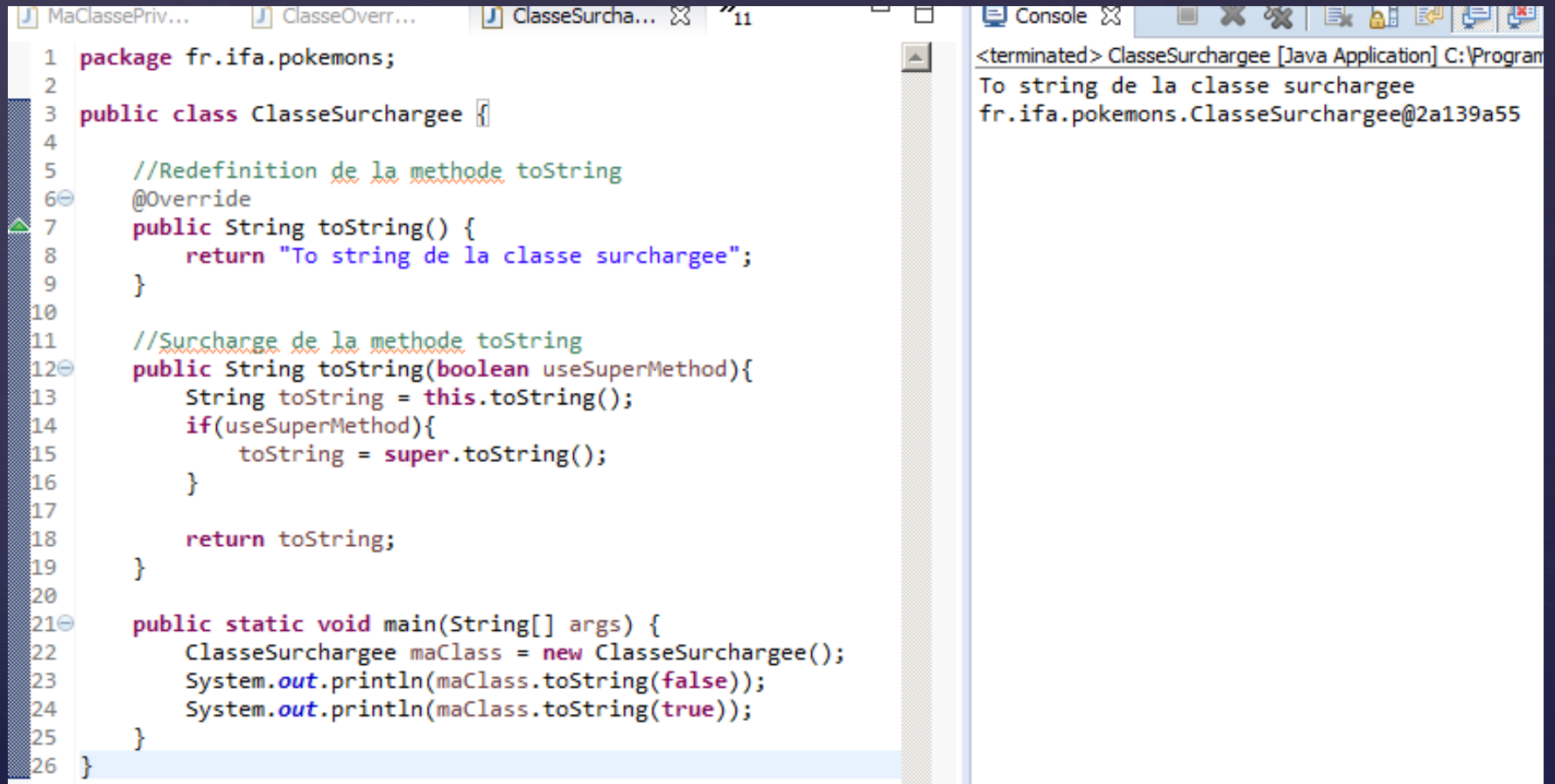
```
1 package fr.ifa.pokemons;
2
3 public class ClasseOverride {
4
5     @Override
6     public String toString() {
7         String toString = super.toString();
8         toString = toString + "Ma classe Override";
9         return toString;
10    }
11
12    public static void main(String[] args) {
13        ClasseOverride classOverride = new ClasseOverride();
14        System.out.println(classOverride);
15    }
16 }
17
```

Console

<terminated> ClasseOverride [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (6
fr.ifa.pokemons.ClasseOverride@2a139a55Ma classe Override

Surcharge des méthodes

Pour accéder à ses données de manières différentes. On peut définir des méthodes qui porte **le même nom** mais avec des **signatures différentes**.



```
1 package fr.ifa.pokemons;
2
3 public class ClasseSurchargee {
4
5     //Redefinition de la methode toString
6     @Override
7     public String toString() {
8         return "To string de la classe surchargee";
9     }
10
11     //Surcharge de la methode toString
12     public String toString(boolean useSuperMethod){
13         String toString = this.toString();
14         if(useSuperMethod){
15             toString = super.toString();
16         }
17
18         return toString;
19     }
20
21     public static void main(String[] args) {
22         ClasseSurchargee maClass = new ClasseSurchargee();
23         System.out.println(maClass.toString(false));
24         System.out.println(maClass.toString(true));
25     }
26 }
```

Console

```
<terminated> ClasseSurchargee [Java Application] C:\Program
To string de la classe surchargee
fr.ifa.pokemons.ClasseSurchargee@2a139a55
```

Surcharge des méthodes à la déclaration

Il est possible de surcharger des méthodes lors de la déclaration.

```
17 public static void main(String[] args) {  
18     Pokemon miaouss = new Pokemon("Miaous",10,30, Type.Normal){  
19         public void attaquer() {  
20             System.out.println("Attaque griffe");  
21         }  
22     };  
23  
24     miaouss.attaquer();  
25 }  
26  
27 }
```

Console

<terminated> Pokemon.1 [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (4 déc. 2016 22:47:41)

Attaque griffe

Liaison statique et dynamique

- Liaison statique = invocation d'une méthode connue à la compilation.
- Liaison dynamique (ou retardée) = invocation d'une méthode connue à l'exécution.

```
1 package fr.ifa.pokemons;
2
3 import java.util.Random;
4
5 public class LiaisonStatiqueDynamique {
6
7     private class Object1 {
8         @Override
9         public String toString() {
10             return "Object1";
11         }
12     }
13
14     private class Object2 {
15         @Override
16         public String toString() {
17             return "Object2";
18         }
19     }
20
21     @Override
22     public String toString() {
23         return "LiaisonStatiqueDynamique";
24     }
25
26     public static void main(String[] args) {
27         LiaisonStatiqueDynamique maClasse = new LiaisonStatiqueDynamique();
28         //Invocation statique
29         System.out.println(maClasse);
30
31         Object objet1 = maClasse.new Object1();
32         Object objet2 = maClasse.new Object2();
33
34         Object myObject = new Random().nextBoolean() ? objet1 : objet2;
35         //Invocation dynamique retardée
36         System.out.println(myObject);
37     }
}
```

<terminated> LiaisonStatiqueDynamic
LiaisonStatiqueDynamique
Object2

Polymorphisme

Le polymorphisme veut dire qu'une méthode peut avoir un comportement différent selon les situations.

Reprenons l'exemple du Pokemon :



```
1 package fr.ifa.pokemons;
2
3 public class Polymorphisme {
4
5     public static void main(String[] args) {
6         //Creation d'un pokemon Pikachu
7         Pokemon pokemon = new Pikachu(45,423);
8         pokemon.attaquer();
9
10        pokemon = new Miaouss(50,343);
11        pokemon.attaquer();
12    }
13 }
```

Console

<terminated> Polymorphisme [Java Application] C:\Program Files\Java\jre1.8

Attaque électrique

Attaque griffe



- ✓ *Les interfaces*
 - *Origine*
 - *Définition*
 - *Déclaration*
 - *Implémentation*
- ✓ *Classe abstraite*
 - *Définition*
 - *Déclaration*

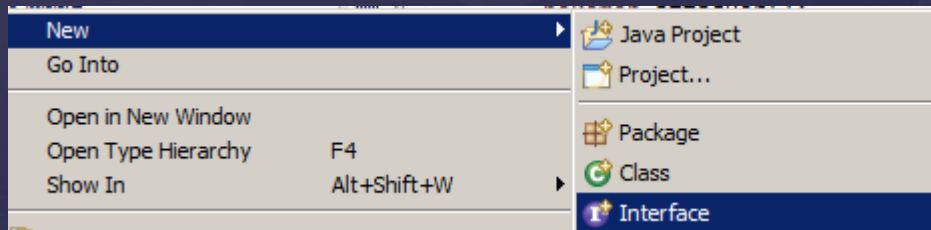
Héritage multiple : en C++, une classe fille peut posséder plusieurs parents. Il présente de nombreux avantages d'un point de vue conceptuel. En effet, il est plus facile de construire ses propres objets en utilisant un travail déjà effectué sur les objets parents. Mais il faut savoir que l'héritage rend les compilateurs soit peu performants soit complexes. En JAVA, **l'héritage multiple n'existe** pas mais les **interfaces** en reprennent les principaux avantages.

Les interfaces traduisent la capacité d'un objet à adopter un comportement en plus de ce que son parent peut lui fournir.

Une interface impose que toutes classes qui l'implémentent doivent implémenter toutes ses méthodes.

Toutes les méthodes d'une interfaces sont **abstraites** et ne possèdent pas de corps. Les méthodes implémentées sont dites **concrètes**.

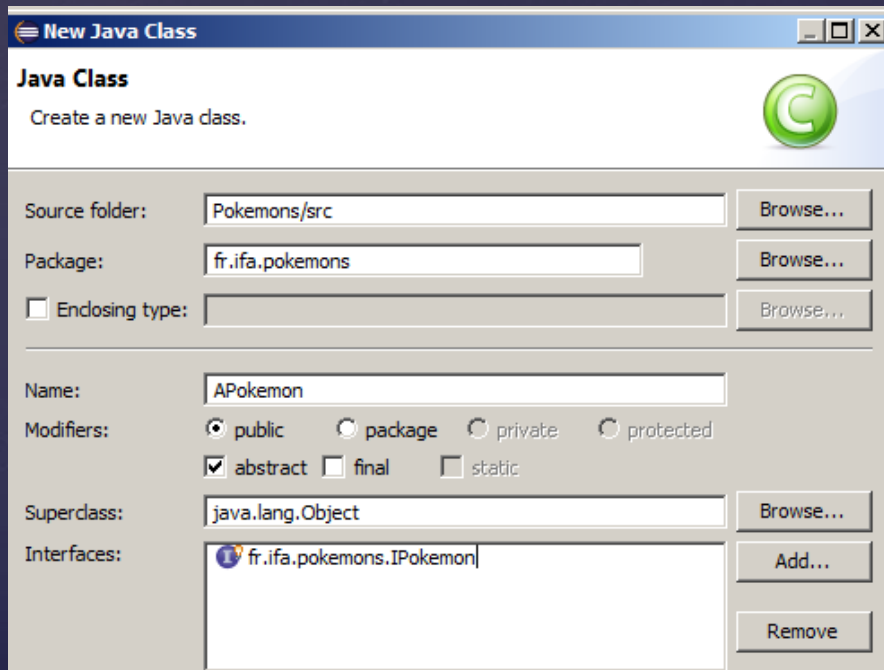
Une interface peut contenir des champs **public**, **static** ou **final**



```
1 package fr.ifa.pokemons;
2
3 public interface IPokemon {
4
5     public void attaquer();
6
7 }
```

```
1 package fr.ifa.pokemons;
2
3 public class Miaouss extends Pokemon implements IPokemon {
4
5     public Miaouss(int pv, int pc) {
6         super("Miaouss", pv, pc, Type.Normal);
7     }
8
9     @Override
10    public void attaquer(){
11        System.out.println("Attaque griffe");
12    }
13 }
```

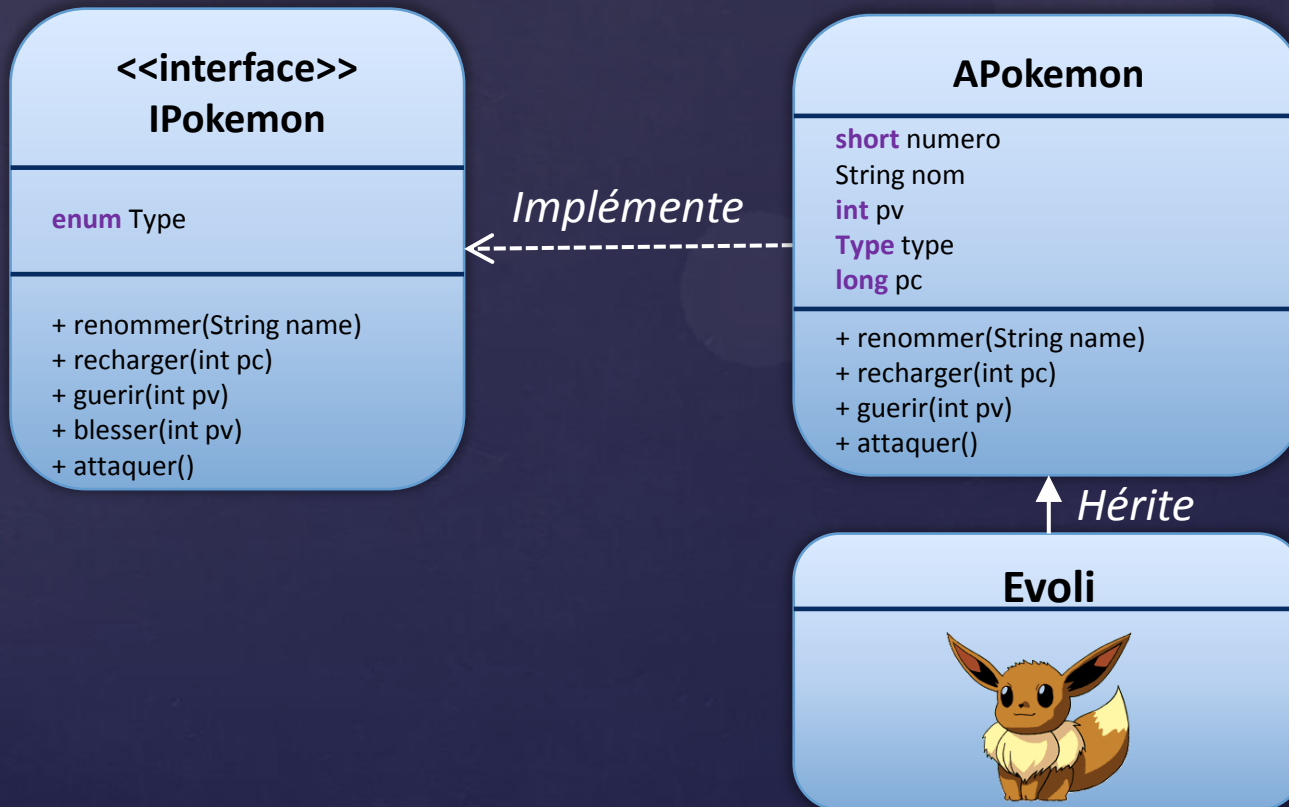
- Une classe abstraite est une classe qui peut contenir des méthodes abstraites et qui n'est pas obligée d'implémenter des méthodes d'interface.
- Elle représente l'avantage de pouvoir définir dans une classe parente un comportement commun aux classes filles y compris dans le constructeur.
- Par contre il est impossible d'instancier une classe abstraite.



```
1 package fr.ifa.pokemons;  
2  
3 public abstract class APokemon implements IPokemon {  
4  
5 }
```

TP n°3 : Pokémon interface, classe abstraite et héritage

- ✓ Créer l'interface IPokemon
- ✓ Créer la classe abstraite APokemon
- ✓ Créer la classe Evoli



Interface IPokemon

```

1 package fr.ifa.pokemons;
2
3 public interface IPokemon {
4
5     public static final Type TYPE_PAR_DEFAULT = Type.Normal;
6
7     // Enumeration type de pokemon
8     public static enum Type {
9         Normal, Electrik, Eau, Feu
10    }
11
12    public void renommer(String name);
13
14    public void recharger(long pc);
15
16    public void guerir(int pv);
17
18    public void attaquer();
19
20 }

```

Classe Evoli

```

1 package fr.ifa.pokemons;
2
3 public class Evoli extends APokemon {
4
5     //Constructeur avec paramètre
6     public Evoli(final int pv, final int pc){
7         super("Evoli", pv, pc, Type.Normal);
8     }
9
10    public void attaquer(){
11        System.out.println("Météor");
12    }
13
14    public static void main(String[] args) {
15        //Creation d'un pokemon Evoli
16        Evoli evoli = new Evoli(76,526);
17        System.out.println(evoli);
18        //Attaque evoli
19        evoli.attaquer();
20    }
21 }

```

Classe abstraite APokemon

```

1 package fr.ifa.pokemons;
2
3 public abstract class APokemon implements IPokemon {
4
5     private String nom;
6     // Point de vie
7     private int pv;
8     // Point de combat
9     private int pc;
10    // Type de pokemon
11    private Type type = TYPE_PAR_DEFAULT;
12    // pvMax
13    private int pvMax;
14
15    //Constructeur avec paramètre
16    public APokemon(final String nom, final int pv, final int pc, final Type type){
17        this.pv = pv;
18        this.pc = pc;
19        this.type = type;
20        this.pvMax = pv;
21        renommer(nom);
22    }
23
24    @Override
25    public void renommer(String nom){
26        if(nom != null && !nom.isEmpty()){
27            this.nom = nom;
28        }
29    }
30
31    @Override
32    public void guerir(int pvPlus) {
33        pv = pv + pvPlus > pvMax ? pvMax : pv;
34    }
35
36    @Override
37    public void blesser(int pvMoins) {
38        pv = pv - pvMoins < 0 ? 0 : pv;
39    }
40
41    @Override
42    public void recharger(int pcPlus) {
43        pc = pc + pcPlus;
44    }
45
46    public String getNom() {
47        return nom;
48    }
49
50    public int getPv() {
51        return pv;
52    }
53
54    public int getPc() {
55        return pc;
56    }
57
58    public Type getType() {
59        return type;
60    }
61
62    @Override
63    public String toString() {
64        String toString = "Nom="+getNom() + "\n";
65        toString = toString + "Type="+getType() + "\n";
66        toString = toString + "Pc="+getPc() + "\n";
67        toString = toString + "Pv="+getPv() + "\n";
68        return toString;
69    }
70 }

```

TP n°4 : Pokémons diagramme de classes

